1. Project Proposal

Title: Web System for Student Hackathon Management with Multi-Criteria Team Formation.

Team Members: Elkham Mamedov, Ilnur Saipiyev, Alisher Askar.

Supervisor: Nurtas Nadirov

Elkham Mamedov: responsible for requirements analysis, QA, integration and documentation.

Ilnur Saipiyev: responsible for the backend, database and matching algorithm.

Alisher Askar: responsible for the frontend.

Project stage: analysis, design and initial technical setup.

Problem Statement

Organizing student hackathons presents significant administrative and algorithmic challenges. On the administrative side, organizers struggle with fragmented workflows, including handling participation registration, team formation, submissions, and evaluations across multiple disconnected tools (e.g., Google Forms, spreadsheets, messaging apps). On the participant side, team formation is often inefficient and biased. Students frequently form teams solely based on existing social connections rather than complementary skills, leading to unbalanced teams. Unaffiliated participants ("solo registrants") face high barriers to entry, often being assigned randomly or left out entirely. The reviewed alternatives do not establish a single workflow combining full hackathon administration, explainable multi-criteria recommendations and participant confirmation. This combination motivates the proposed design.

Target Users

- Hackathon organizers, including academic staff, student organizations, or event administrators who manage event schedules, registration, team allocations, announcements, and scoring/judging workflows
- Student participants, whether individual students or pre-formed teams looking for missing skill sets, project tracks, or suitable teammates based on technical criteria.

- Judges, mentors, or faculty members, who will be evaluating project submissions based on specific scoring rubrics.

Aim

To design, implement, and evaluate a centralized web platform that streamlines end-to-end hackathon administration while automating balanced, skill-based team formation via a multi-criteria algorithm.

Objectives

1. Requirements and Research. Conduct user interviews and comparative analyses of existing hackathon platforms and matching algorithms to establish system requirements.
2. System Architecture and Design. Design a scalable web application architecture with database schemas, API contracts, and user interaction scenarios.
3. Multi-criteria matching algorithm. Formulate and implement a team formation algorithm that processes participant profiles (skills, roles, project preferences, availability) to suggest or create balanced teams.
4. Platform core features development. Implement participant profile management, organizer management dashboards, real-time team discovery, and submission/scoring modules.
5. Testing and Evaluation. Validate system performance, usability, and matching algorithm quality against a baseline solution using clear evaluation metrics.

Expected Outcomes

- A fully operational web platform with responsive user interfaces and scalable RESTful APIs.

- An algorithmic engine capable of generating multi-criteria match scores for team formation.
- Unified handling of event creation, registration, team formation, project submission, and judging.
- Complete technical and analytical documentation, including architecture diagrams, literature reviews, technology comparisons, and verification plans.

Project Scope

In-Scope

- User Management: authentication, authorization, role-based control, and detailed skill/profile building.
- Hackathon event management: event creation, track setting, registration timelines, schedule updates, and announcements.
- Multi-criteria team formation:
    - Algorithmic teammate recommendations for incomplete teams.
    - Auto-grouping mechanism for solo registrants based on complementary skill matrices and role coverage.
- Submission and evaluation portal: project submission links (repository, demo), assigned judging scoring sheets, and leaderboard generation.
- System documentation and testing: source code repository management, task traceability, end-to-end testing, and architectural design.

Out-of-Scope

- Native mobile applications. The system will be implemented strictly as a responsive web application.
- In-platform live code execution / sandbox. Code testing and verification will rely on external links (e.g., GitHub, live demo URLs).

- Payment / prize distribution integrations. Financial transactions or sponsorship payment processing will not be included in the initial implementation.

2. Evidence of the Relevance

The primary data for the project come from the team's Student Hackathon Survey, collected through Google Forms. The response summary contains 17 submissions: 13 respondents consented to participate and four declined. Questions 2 to 11 each contain 13 responses, and Q12 contains five optional text responses. The analysis uses the consenting responses. Counts and percentages below come from the corresponding question summaries, preserved as screenshots in the project repository. The questionnaire link provides the questions; the repository screenshots provide the results used in this report. For questions allowing several selections, percentages use 13 respondents as the denominator and need not sum to 100%.

In Q2, six of the 13 respondents reported no hackathon experience, three reported one event, two reported two to three events, and two reported four or more. Non-participation totals differ between Q2, Q3 and Q6. The small sample and inconsistent answers limit generalisation; question-specific totals are reported without calculating experienced-only rates.

In Q10, six respondents (46.2%) preferred choosing teammates themselves, three (23.1%) preferred multi-criteria suggestions with a final personal choice, and one (7.7%) preferred system assignment with an option to request changes. No respondent selected organiser assignment. These preferences support voluntary recommendations and participant confirmation, but do not establish that a matching algorithm improves team performance.

In Q11, registration and profiles received seven selections (53.8%), judging criteria and feedback five (38.5%), and teammate search and announcements four each (30.8%). These choices identify the functions respondents considered useful and support bringing registration, team discovery, event information and judging into one platform. Q8 also recorded two selections each for schedule changes and confusing submission, and one each for scattered

information and unclear rules or judging. These are reported concerns within this sample, rather than evidence that every respondent experienced these problems.

These findings justify developing a focused HackMatch prototype with registration and profiles as core functions, visible judging criteria and feedback, shared announcements and optional teammate recommendations. Q10 supports keeping the final team decision with participants: six respondents preferred choosing teammates themselves and three preferred suggestions followed by personal choice. The proposed workflow will therefore let participants review recommendations and accept or decline them. The survey establishes priorities for implementation; the evaluation in Section 9 will test whether the resulting system is usable and whether matching meets its stated criteria.

Q7 recorded three selections for different commitment levels and two for unclear roles. These responses support making commitment preferences and team roles visible in participant profiles. Two Q12 responses suggested pre-event networking or a central platform, which supports providing a shared place for participant discovery and event activities. Q7 also contains overlapping no-difficulty and not-applicable selections, so these findings are exploratory. No respondent selected missing skills as an experienced difficulty in Q7. Complementary skills are therefore a design criterion informed by the literature in Section 3, rather than a problem demonstrated by this survey.

The reason to develop this project is the combination of these locally identified priorities with the gap discussed in Sections 3 and 4: a student hackathon workflow that includes event administration, participant profiles, explainable teammate recommendations and participant confirmation. The survey alone does not establish that existing platforms cannot meet these needs or that the proposed algorithm improves team performance. The platform comparison provides the basis for the development gap, while prototype testing and algorithm evaluation will assess the proposed solution.

Questionnaire:

https://docs.google.com/forms/d/e/1FAIpQLSeZeYzaP4SF2Z0Tic4WnJDYXq45ePErZ1lqkmP_JJAII4qxSQ/viewform

Survey results and supporting screenshots: Опросник folder in the project repository

3. Literature Review

The literature review covers twelve academic publications about team formation, student collaboration and hackathon coordination. The studies differ in their settings and methods, so their findings should not be treated as directly interchangeable. Hastings et al. (2023) studied instructors' use of algorithmic team formation and showed why criteria and their effects should be understandable. Forshaw et al. (2026) examined fairness in student group formation and the trade-offs involved in different grouping approaches. Agarwal et al. (2022) explored genetic optimisation for balancing student competencies. Singh et al. (2025) used multi-criteria optimization to study team diversity and potential conflict. These studies inform the design of a matching algorithm, but technical balance does not automatically establish successful collaboration. Wu et al. (2026) considered perceived pressure in heterogeneous student grouping, while Xu et al. (2026) examined personality and collaboration in design education. Their settings differ from short student hackathons. Vinella et al. (2022) compared bottom-up, top-down and hybrid team formation in simulations, suggesting that participant choice deserves attention. Gomez-Zara et al. (2019) investigated online collaborator discovery and the role of existing social connections. Kohli et al. (2023) studied opt-in study-group matching in a large remote course. Jadhav et al. (2026) presented ResearchConnect, integrating profiles and research team suggestions. Hevenstone et al. (2026) discussed self-assigned and algorithmically assigned hackathon teams, including the possibility of structured changes. Mendes et al. (2022) examined collaboration problems in online hackathons. Research gap: existing studies and tools address algorithmic grouping, collaborator discovery or event coordination separately. This project will investigate how transparent multi-criteria suggestions

can be combined with a complete student hackathon workflow and evaluated through measurable tasks. The project does not claim that multi-criteria grouping itself is new.

Author & Year	Topic / Focus	Methodology	Relevance to Project
Hastings et al. (2023)	Instructor algorithmic team grouping	Survey of 77 instructors and 21 interviews	Supports transparent grouping criteria and understandable controls
Forshaw et al. (2026)	Fairness in student group formation	Review of 262 papers, 18 tools, interviews and focus groups	Frames fairness trade-offs and allocation constraints
Agarwal et al. (2022)	Balancing engineering student competencies	Vector Evaluated Genetic Algorithm; ~1,300 students	Informs weighted skill and role balancing
Singh et al. (2025)	Multicriteria team formation	Clustering and multicriteria integer programming; simulated profiles	Informs explicit role and diversity constraints
Wu et al. (2026)	Pressure in heterogeneous groups	81-student class; NSGA-II grouping analysis	Highlights social effects beyond technical matching
Xu et al. (2026)	Personality in design student teams	Regression analysis; 50 students in 25 two-person teams	Motivates measuring collaboration separately from scores
Vinella et al. (2022)	Worker-centered team formation	Agent-based comparison of bottom-up, top-down and hybrid groups	Supports opt-in choice and hybrid matching
Gómez-Zará et al. (2019)	Online collaborator discovery	Field study; 43 participants and eight teams	Addresses discovery and existing social connections
Kohli et al. (2023)	Inclusive study group matching	Opt-in matching study in a large remote course	Informs opt-in matching for solo participants
Jadhav et al. (2026)	ResearchConnect profiles and ideation	Profile clustering; 250 research summaries	Related profile-based recommendations outside hackathons
Hevenstone et al. (2026)	Diversity and participant involvement	Comparison of two civic hackathons	Motivates participant agency and adjustments

Mendes et al. (2022)	Virtual hackathon coordination	Study of ten teams across four online hackathons	Motivates unified event and collaboration workflows

Empirical studies and experiments

The reviewed evidence includes computational experiments using real student data, a deployed matching system, observational field studies and a simulation experiment. Their methods, findings and limitations inform different parts of the proposed design.

Agarwal and colleagues 2022 computational experiment with student data

Agarwal et al. (2022) used self-reported programming, spreadsheet and general computer competencies from approximately 1,300 first-year engineering students to form teams with a Vector Evaluated Genetic Algorithm. The paper compared algorithmic and manual allocations and reported a more uniform distribution of team-average skills. In one 47-student example, 100 generations with 1,500 candidate configurations took about 18 seconds. The algorithm prioritised programming when it could not minimise all objectives simultaneously. This result demonstrates a practical allocation method, but does not establish that the generated teams collaborated or learned better; evaluation of team effectiveness remained future work. For this project, the study motivates comparison against manual allocation, explicit reporting of runtime and measurement of trade-offs between criteria.

Kohli and colleagues 2023 deployed study group matching

Method and setting. The authors trialled opt-in software matching in an introductory engineering and computer science course with more than 1,000 students during remote teaching. Students could request reassignment. Results. Underrepresented students were more likely to request matching; matched students reported comparable group experiences across backgrounds, and positive group experiences were associated with exam outcomes. Limitation. This was a course deployment, not a randomised student hackathon experiment; the associations do not prove a causal benefit of our proposed algorithm. The findings support

voluntary participation, reassignment and separate measures of comfort and activity in HackMatch.

Mendes and colleagues 2022 four real online hackathons

Method and sample. A multiple-case study examined 10 teams across four online hackathons on two continents, using 27 interviews. Results. Teams combined synchronous and asynchronous tools; responsibilities were often self-selected by skill or interest. Networking and informal mentor access decreased, while multiple tools created technical and information burdens. Limitation. This observational qualitative study did not compare matching algorithms or quantify a causal effect of a central platform. These findings inform the central event workflow, visible team roles and end-to-end evaluation in HackMatch.

Hastings and colleagues 2023 instructor use of grouping algorithms

Method and sample. The researchers surveyed 77 instructors and conducted 21 follow-up interviews about criteria-based grouping, including CATME. Results. Instructors checked proposed teams against educational goals but sometimes struggled to translate those goals into tool settings. Student input was often absent. Limitation. The work describes instructor practice rather than experimentally proving higher team performance. These findings support visible matching criteria, organiser review and participant feedback in HackMatch.

Vinella and colleagues 2022 simulation experiment

Method. The authors simulated bottom-up, top-down and hybrid team formation. Results. Self-organisation favoured the strongest teams, while the hybrid model better protected the weakest teams; mean quality did not differ significantly across models. Limitation. Outcomes were generated by an agent model and its assumptions, not observed in a live hackathon. The HackMatch evaluation therefore includes minimum and average role coverage and unassigned participants, alongside the strongest team outcomes.

4. Existing system comparison

Devpost provides hackathon listings, project submissions and participant discovery. Its participant documentation describes ways to find teammates, but does not establish a weighted automatic allocation process based on skills, interests and availability. Devpost for Teams allows projects to advertise roles and participants to request membership.

Devfolio supports event participation and team creation or joining, including invitation-based workflows. The reviewed guidance does not establish multi-criteria automatic grouping. CATME Team-Maker collects student survey information and allows instructors to configure grouping criteria, including skills and schedules. However, it focuses on educational team allocation rather than full hackathon administration. gruepr also supports survey-based team formation, constraints and manual adjustments, but its main focus is educational grouping.

My Dream Team, studied by Gomez-Zara et al. (2019), explored profile-based collaborator discovery. ResearchConnect (Jadhav et al., 2026) integrates researcher profiles, clustering and ideation, but its evaluation does not demonstrate deployment in a live student hackathon.

The proposed system will combine event registration, profile-based suggestions, participant confirmation, organiser adjustments, announcements and submissions. These functions are planned rather than completed. Its usefulness must be evaluated against the existing alternatives and a simpler manual baseline.

Comparison Criteria	**Devpost**	**Devfolio**	**CATME**	**gruepr**	**Proposed System**
Hackathon event workflow	Yes	Yes	No	No	Planned
Participant skill profiles	Basic	Basic	Survey-based	Survey-based	Planned
Multi-criteria grouping	Not established	Not established	Yes	Yes	Planned
Weighted skill	Not	Not	Configurable	Configurable	Planned

matching	established	established	criteria	criteria	
Explainable suggestions	Not established	Not established	Instructor settings	Organizer settings	Planned
Solo registrant support	Discovery/manual	Join/invite	Instructor-led	Instructor-led	Planned
Manual adjustment	Team management	Team management	Instructor controls	Instructor controls	Planned
Project submissions	Yes	Yes	No	No	Planned

Justification for developing the proposed system

The proposed project addresses a specific combination of needs in student hackathons: managing the event workflow and helping participants form teams while preserving their freedom of choice. In the reviewed comparison, hackathon platforms mainly support event participation and project submission, whereas educational team-formation tools focus on allocating students to groups. The proposed system brings these functions into one workflow: participant profiles provide information for team suggestions, accepted suggestions become event teams, and those teams proceed to project submission and judging.

Our preliminary survey provides a local reason for this design. Of the 13 consenting respondents, seven selected registration and participant profiles as useful features, five selected judging criteria and feedback, and four selected teammate search and suggestions. Six respondents preferred choosing teammates themselves, while three preferred receiving suggestions and making the final decision personally. These findings support a platform that offers optional recommendations alongside manual team formation. However, the small sample and inconsistent answers require further validation with participants and organisers.

The distinctive contribution is the integration of explainable, multi-criteria team recommendations into the complete student hackathon workflow. Recommendations will consider complementary skills, preferred roles, interests and availability, explain the criteria used, and allow participants to accept or decline. Organisers will be able to review allocations and resolve cases where team constraints cannot be satisfied.

The development involves more than combining registration forms and dashboards. It requires implementing and evaluating a matching strategy, detecting infeasible constraints, maintaining consistent team membership during simultaneous invitations, and enforcing permissions across participants, organisers and judges. Separate forms, spreadsheets and messaging tools do not automatically enforce these connected rules. A dedicated implementation provides a consistent data model and lets the team compare matching strategies under the same conditions.

A dedicated implementation is justified by the need to connect event-scoped profiles, team constraints, participant decisions and submission permissions in a consistent data model. A configurable matching module also permits a controlled comparison of rule-based and weighted recommendations. The evaluation will compare the complete workflow with manual administration and measure whether this integration reduces effort while maintaining valid teams and participant choice.

5. Technology comparison

The selected stack is Next.js with React, TypeScript and Tailwind CSS for the frontend, Node.js with Express for the REST API, and PostgreSQL for storage. React supports reusable registration forms and dashboards; a JavaScript and TypeScript stack suits the team's development experience. PostgreSQL supports the relationships and transactional updates required for registrations, invitations and memberships. The matching module separates a rule-based baseline from weighted recommendations so both can be evaluated on the same data. The comparison below records the alternatives and reasons for selection. GitHub supports version control, YouTrack task tracking, Figma interface design and Postman API testing.

Technical Decision	**Alternative A**	**Alternative B**	**Alternative C**	**Selected Choice & Justification**
Frontend framework	React: reusable UI; suited to	Vue: component-base	Vanilla JS: lightweight; more	React (Next.js). Reusable components suit dashboards,

	dashboards	d; another viable option	manual UI state	and our frontend developer already has experience with this stack
Backend framework	Django REST: integrated APIs and admin	Node.js / Express: JavaScript across stack	Go: efficient API; additional learning cost	Node.js / Express. JavaScript across the whole stack simplifies teamwork, and Express is quick to set up for a REST API
Database	PostgreSQL: relational and multi-user	MongoDB: flexible documents; complex relations	SQLite: simple local prototype	PostgreSQL. The data is relational (users, events, applications, teams) and needs reliable multi-user access
Matching strategy	Weighted multi-criteria: flexible scoring	Rule-based: interpretable baseline	Manual allocation: evaluation comparator	Weighted multi-criteria. It gives flexible scoring of skills, experience, roles and interests. A rule-based version is built first as a baseline, and manual allocation is used for comparison in the evaluation

6. Requirements Specification

Functional requirements: organisers should create and manage hackathons, set event rules, review registrations, publish announcements, inspect teams and evaluate submissions. Students should register, maintain skill and interest profiles, indicate preferred roles, review team suggestions and confirm or decline membership. The matching module should apply team-size constraints, consider complementary skills and explain suggestions.

Non-functional requirements: responsive pages, role-based access, secure storage of personal information, clear validation errors, reliable submission handling and understandable explanations of matching criteria. When no feasible team exists under the configured constraints, the system should report the issue rather than silently ignore the rule.

ID	**Requirement**	**Priority**	**Verifiable acceptance criterion**
FR-01	Create hackathon events	High	The organizer saves an event with title, dates, rules and team size; it appears in the event list.
FR-02	Register for an event	High	Eligible students submit a registration and

			see confirmation; duplicate submission is blocked.
FR-03	Edit participant profile	High	Students have skills, experience, preferred roles and interests; values remain after login.
FR-04	Suggest teammates	High	System returns candidate teams using specified criteria and configured minimum/maximum size.
FR-05	Confirm or decline a team	High	Students confirm or reject an invitation; team membership updates accordingly.
FR-06	Manage teams manually	Medium	The organizer can reassign a participant; the updated team remains within valid constraints.
FR-07	Submit a project	High	Team submits repository and demo links before deadline; submission is visible to the organizer.
FR-08	View judging results	Medium	An authorized judge submits rubric scores and the organizer sees the results.
NFR-01	Role-based access	High	A participant account cannot reach organizer-only APIs; unauthorized requests return 403.
NFR-02	Responsive interface	High	Registration and team screens are usable at 375 px and 1280 px widths without horizontal overflow.
NFR-03	Explainability and invalid constraints	High	Every suggestion shows criteria used; impossible constraints produce a clear warning instead of an invalid team.

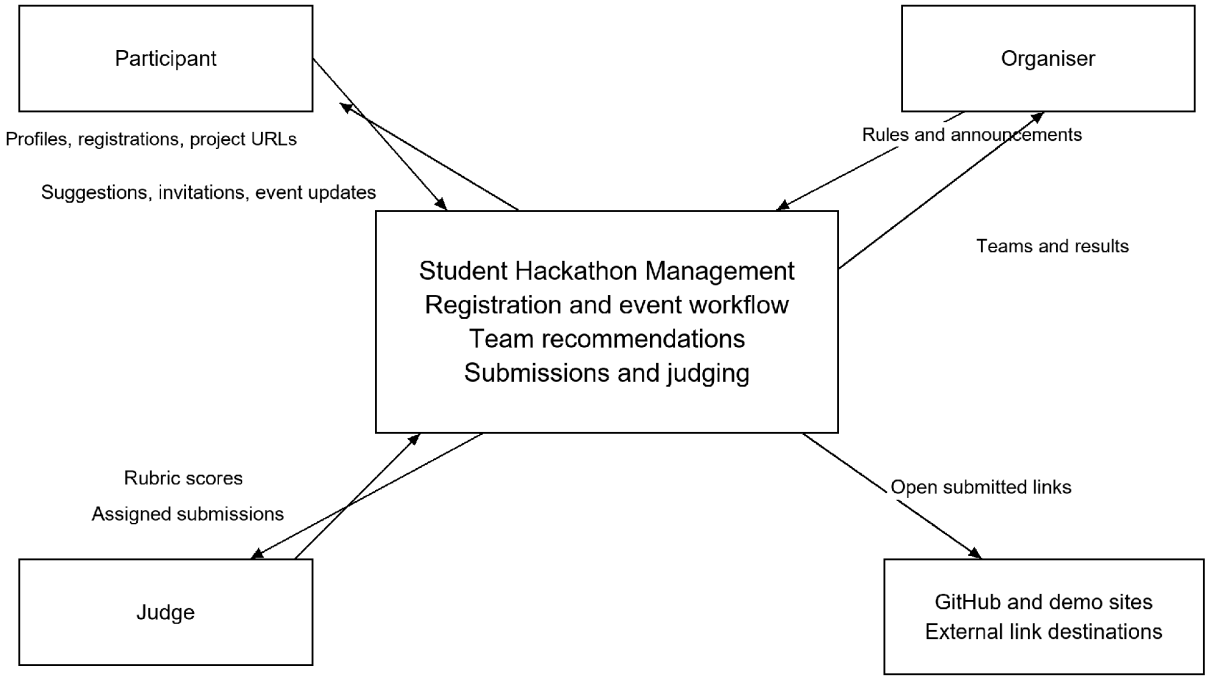
7. Architecture and Diagrams

The proposed system is a three-tier web application. A Next.js frontend built with React, TypeScript and Tailwind CSS communicates with a Node.js and Express REST API. PostgreSQL stores event-scoped records. The matching engine is a separate internal backend module with interchangeable rule-based and weighted strategies. The diagrams describe the proposed system design.

System context diagram

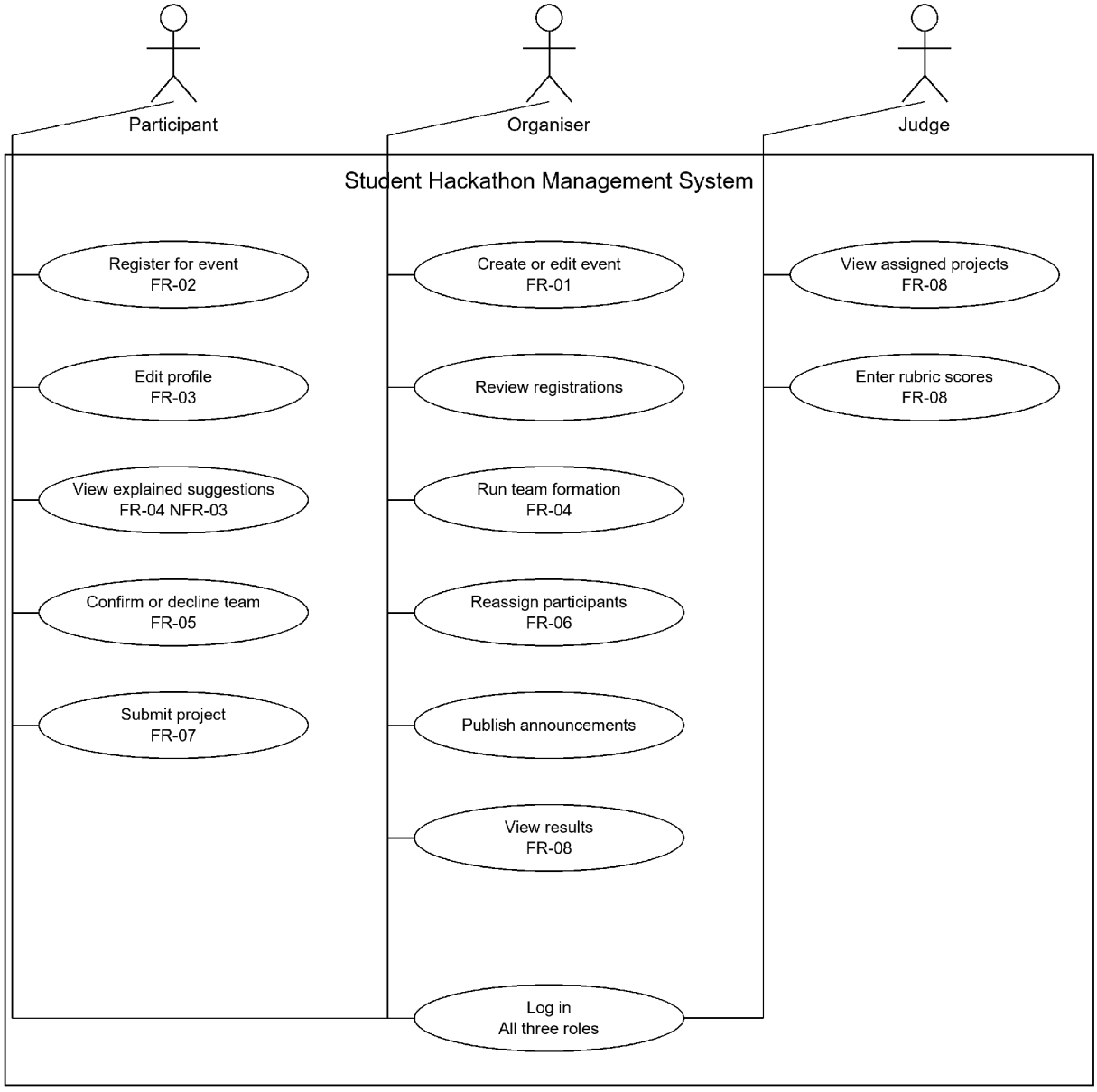
Participants provide profiles, registrations and project URLs. Organisers configure events and team constraints. Judges access assigned submissions and enter scores. GitHub

and demo pages are external link destinations; the system stores URLs and does not execute submitted code.



Use Case Diagram

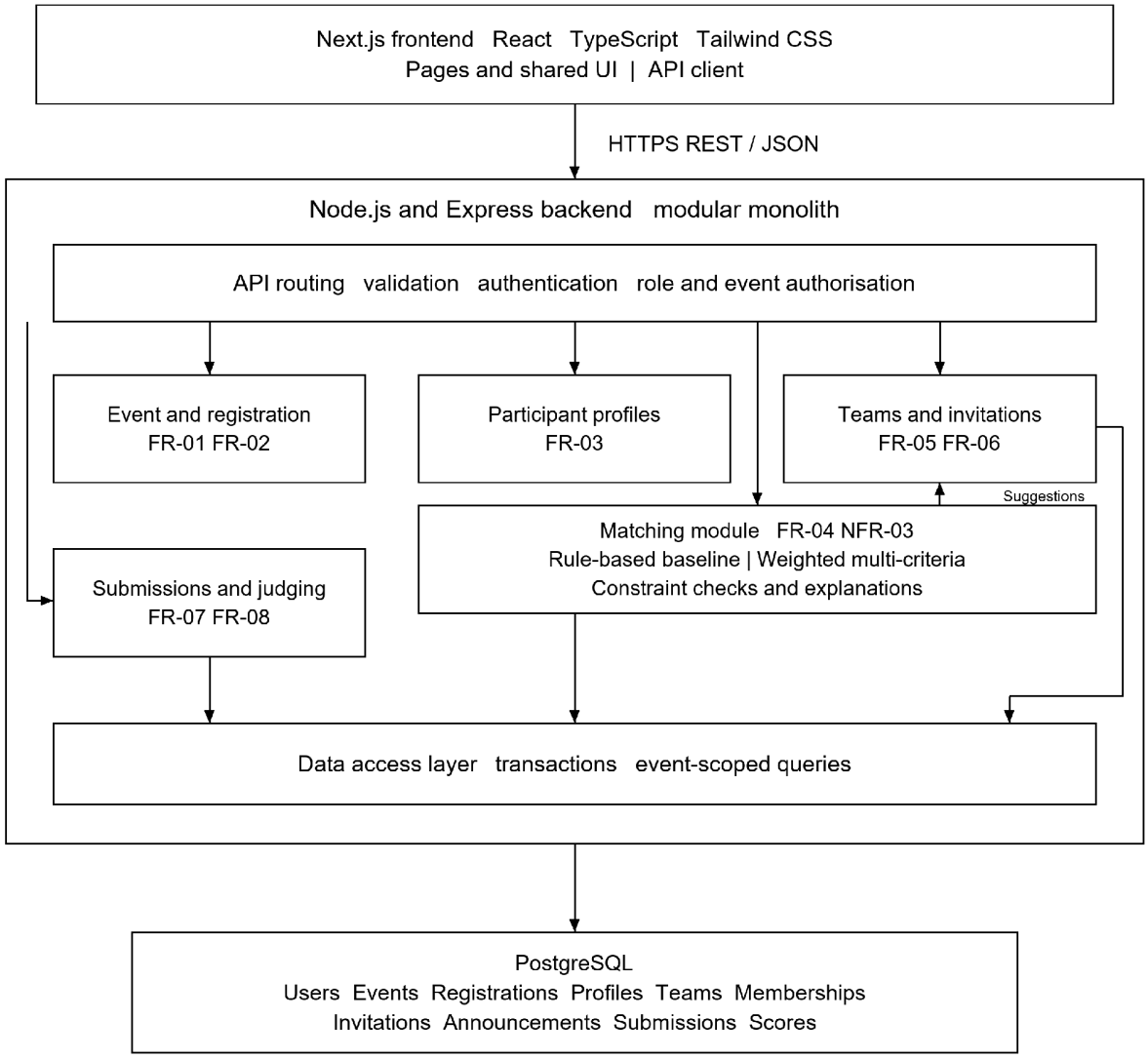
The diagram shows participant, organiser and judge actions. FR-01 to FR-08 refer to the requirements in Section 6. Login is shared, and protected operations enforce both role and event permissions. Event review and announcements support the event management scope.



Component Diagram draft

The API validates requests and enforces permissions before invoking domain modules. Matching reads profiles and event constraints and returns suggestions with explanations. Team

management applies accepted changes. The data access layer uses transactions to prevent duplicate membership and overfull teams.



Alignment. FR-01 and FR-02 map to event and registration management; FR-03 to profiles; FR-04 and NFR-03 to matching and explanations; FR-05 and FR-06 to invitations and team management; FR-07 and FR-08 to submissions and judging. NFR-01 is enforced by access control, and NFR-02 by responsive frontend pages. Stored entities include users, events, registrations, profiles, teams, memberships, invitations, announcements, submissions and scores.

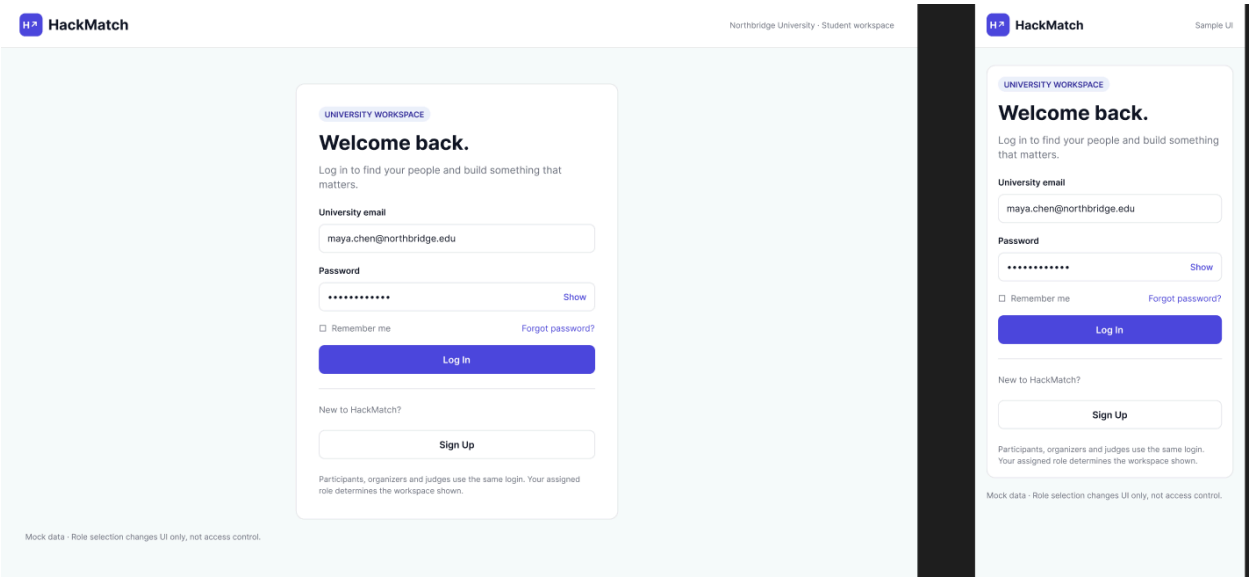
8. UX / Prototype

The HackMatch interface prototype presents desktop and mobile layouts for participants, organisers and judges. The main participant scenario is account creation, profile setup, event discovery, review of explained team suggestions and confirmation or rejection of an invitation. The organiser reviews team composition and can reassign participants; the judge evaluates assigned projects using a shared rubric. The mockups use sample data and illustrative matching explanations.

Figma prototype: HackMatch UI/UX proto

8.1 Login

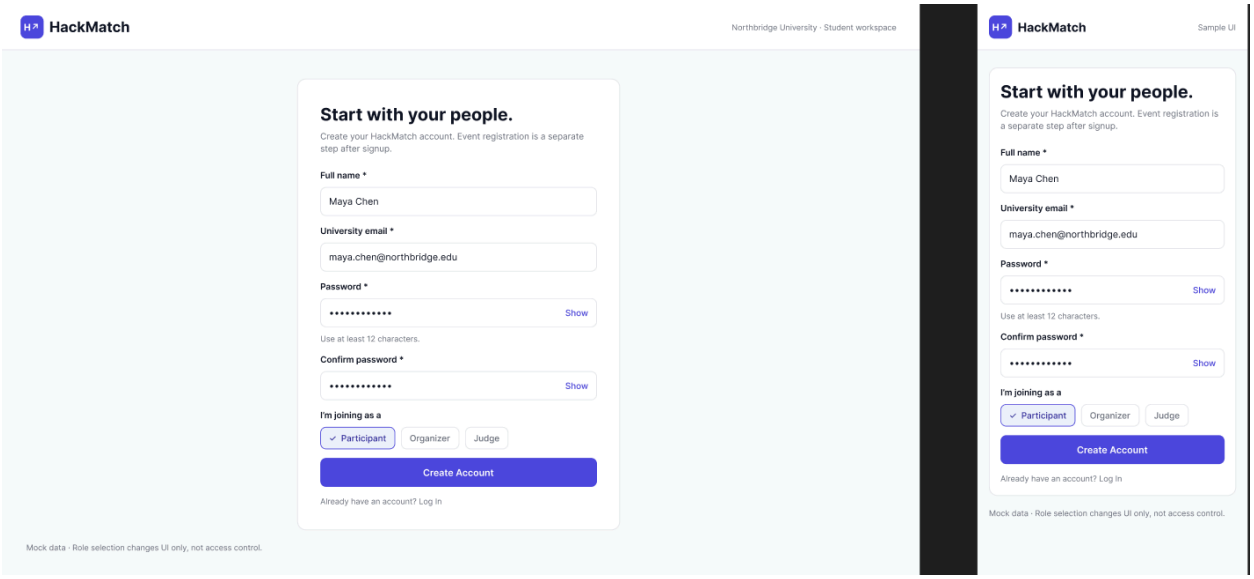
Participants, organisers and judges use a shared login screen. The interface provides university email and password fields, password visibility, account recovery and access to registration. The assigned role determines the workspace displayed after authentication (NFR-01).



8.2 Account registration

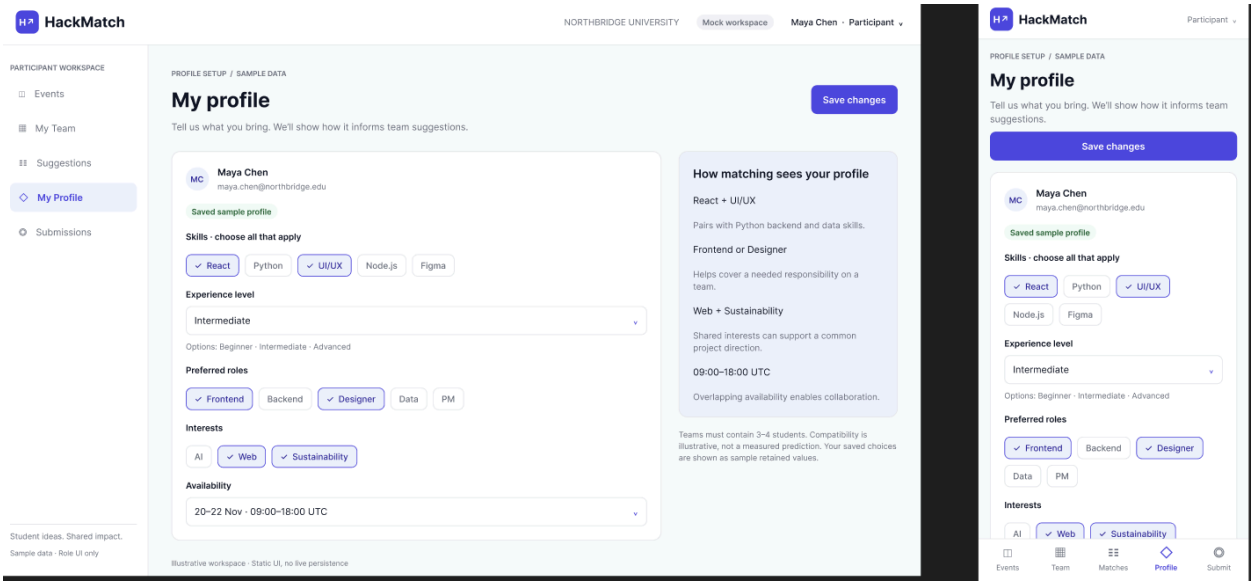
The registration form collects a full name, university email, password and password confirmation. Account creation is separate from registration for a particular hackathon. The role

selector illustrates the available workspaces; organiser and judge access requires server-side authorisation (NFR-01).



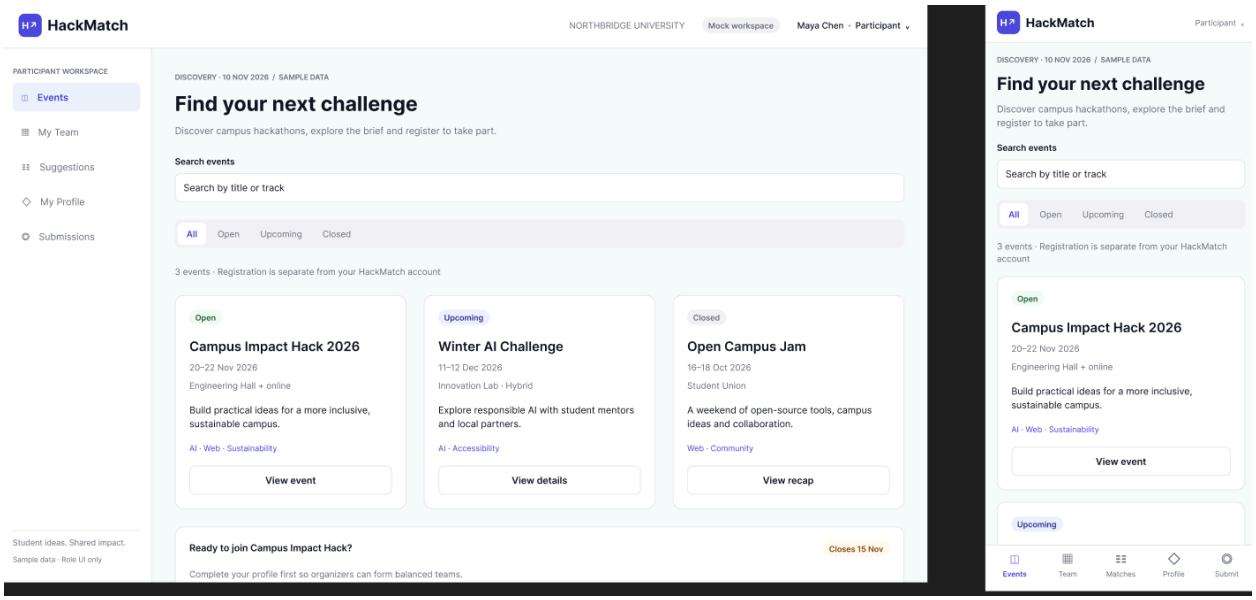
8.3 Participant profile

The profile screen collects skills, experience level, preferred roles, project interests and availability. A summary explains how these attributes inform teammate suggestions. Desktop and mobile layouts present the same profile fields and a save action (FR-03, NFR-02).



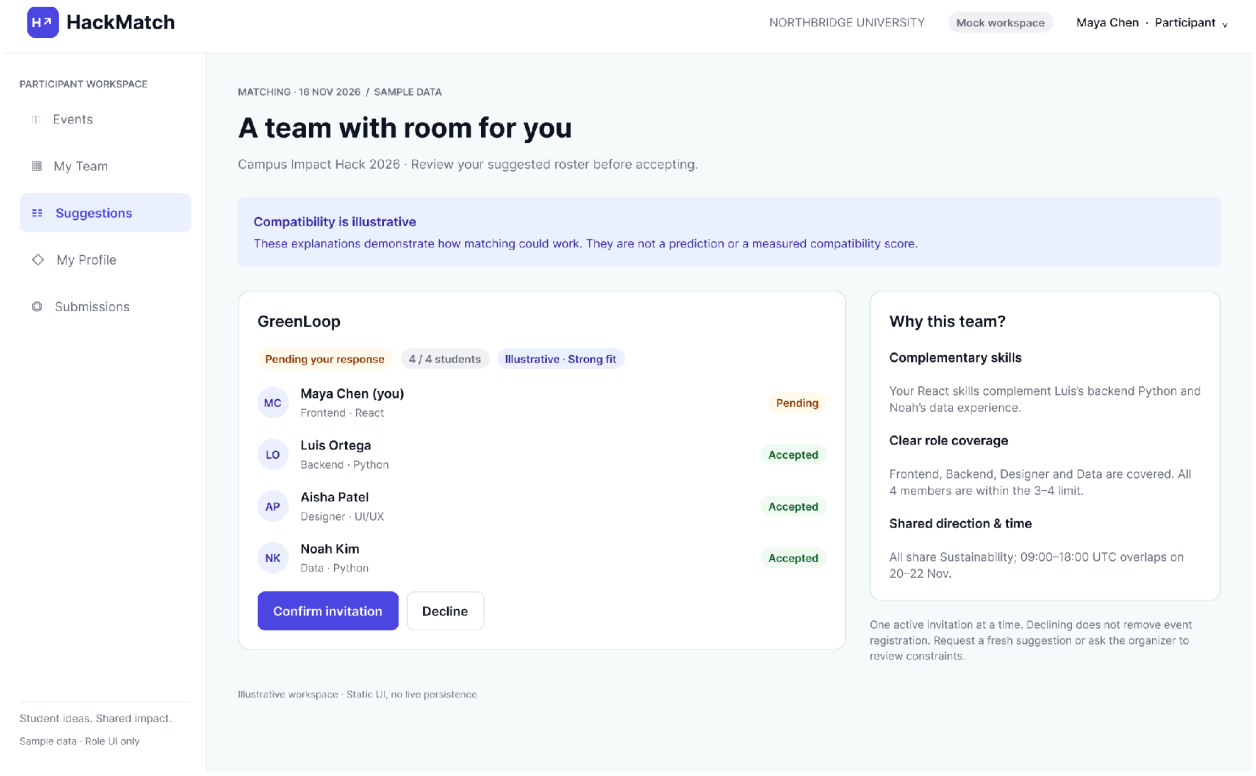
8.4 Hackathon discovery

Participants search events by title or track and filter them by status. Event cards show dates, location or format, a short description and relevant tracks. Opening an event is the entry point to reviewing its rules and registering for participation (FR-02, NFR-02).



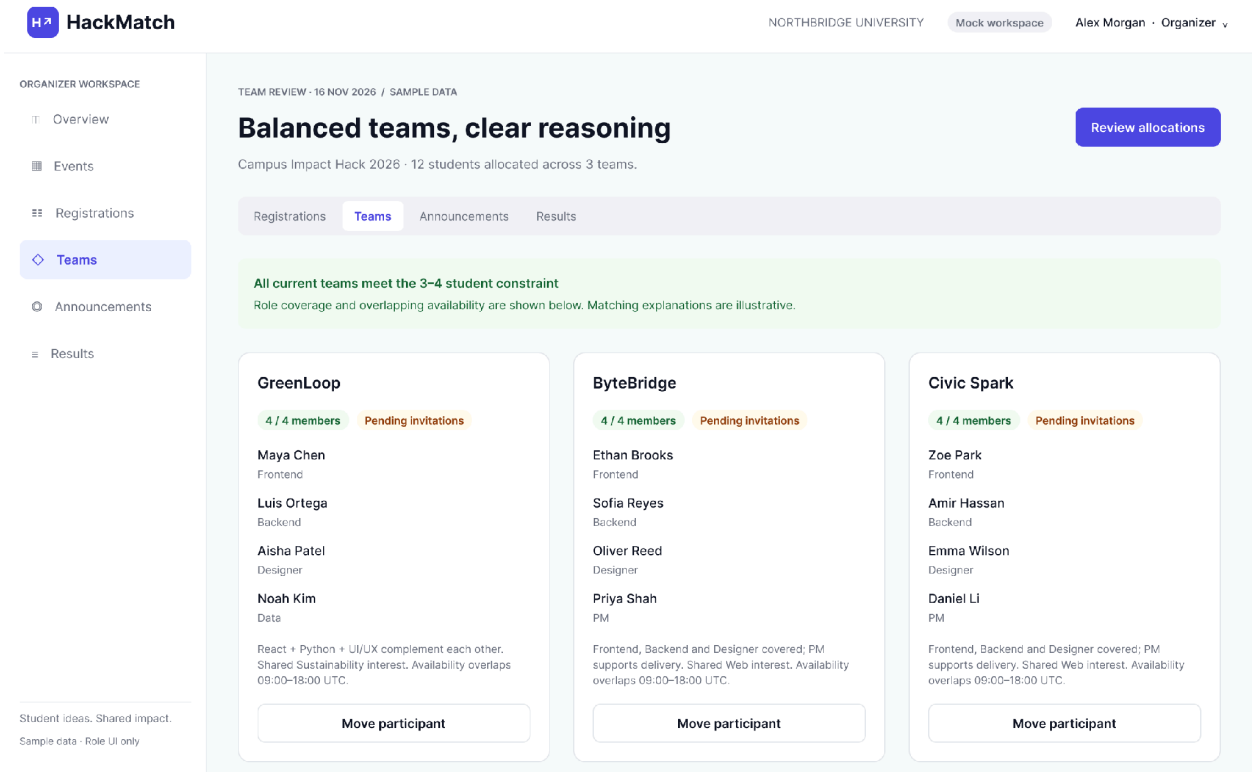
8.5 Team recommendations and confirmation

The recommendation screen presents a proposed roster, member roles, invitation status and an explanation based on complementary skills, role coverage, interests and availability. The participant can confirm or decline the invitation. The displayed fit label is illustrative and is not a measured prediction of team success (FR-04, FR-05, NFR-03).



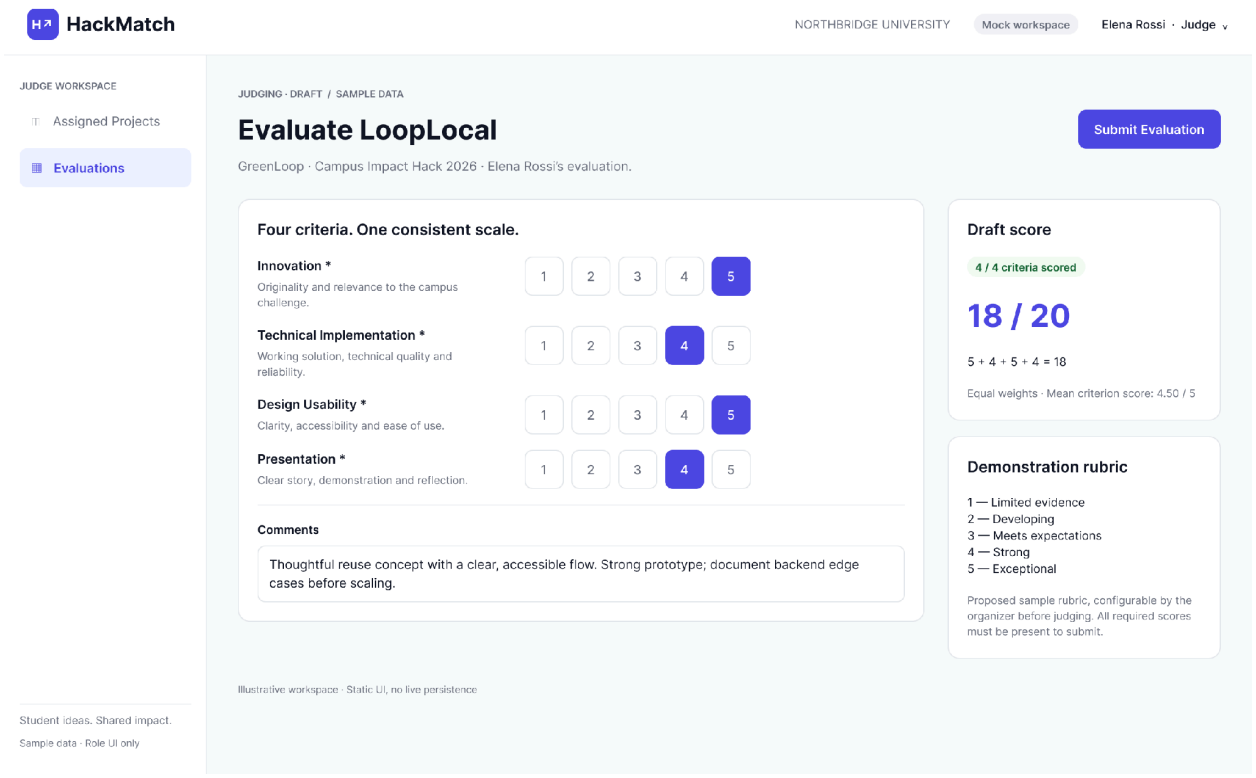
8.6 Organiser team management

The organiser reviews teams, member roles, pending invitations and compliance with the configured team-size limit. Each team provides a control for moving a participant. The design supports reviewing allocations and making manual adjustments while retaining valid membership constraints (FR-06).



8.7 Project evaluation

The judge evaluates an assigned project against four rubric criteria, enters comments and reviews the calculated draft total before submitting an evaluation. The example uses an equal-weight scale from 1 to 5; the visible scores illustrate the interface rather than actual project results (FR-08).



9. Verification and Evaluation Plan

Evaluation will compare allocation quality, task completion and technical correctness. The student questionnaire provides preliminary preference data. Algorithm benchmarks and usability trials are planned after implementation.

Algorithm comparison protocol

The algorithm comparison will use a versioned profile dataset with identical team-size, availability and role constraints for all methods. It will compare deterministic rule-based allocation, weighted multi-criteria allocation and organiser allocation where an organiser is

available. Random allocation will provide an additional comparator over 30 recorded seeds. Real consented profiles and synthetic stress cases will be evaluated separately. Questionnaire responses describe preferences and will not be treated as complete matching profiles.

Benchmark records will include dataset size, hardware, code revision, weights, random seeds and runtime. Outcome measures are valid membership assignments, duplicate memberships, unassigned participants, required-role coverage, minimum team coverage, availability overlap and elapsed formation time. Mandatory-constraint violations will be counted separately from preference scores. Role coverage is the proportion of required roles filled, averaged over teams. The unassigned rate is the proportion of eligible participants left without a team. Organiser effort will be measured separately from algorithm execution time.

The evaluation targets are zero duplicate memberships and zero mandatory-constraint violations on feasible cases, with an explicit unresolved status for infeasible cases. Weighted matching is expected to preserve the assigned-participant count and improve mean role coverage relative to the rule-based baseline. Minimum team coverage and any trade-offs will be reported alongside averages.

Usability and technical verification

A formative usability trial will involve 5-8 consenting students and at least one organiser. Participant tasks will cover registration, profile editing, understanding a suggestion, accepting or declining a team, and project submission. Organiser tasks will cover event creation and resolution of a team issue. Measures will include unaided completion, completion time, errors, requests for help and clarity ratings from 1 to 5. The formative target is at least 80% unaided task completion with no unresolved critical blocker. The small trial will identify usability problems rather than estimate population-wide effectiveness.

Each functional requirement will be checked through positive and negative API tests. Cases include duplicate registration, cross-event access, participant access to organiser APIs, concurrent team confirmations, full teams, impossible constraints, late submissions and judge

access to unassigned projects. Interface checks will use the mobile and desktop widths in NFR-02. Defects and regression results will be recorded in YouTrack. Research records will include consent, collection dates, recruitment channels and question-specific response totals.

10. Project Management and Risks

Project documentation is maintained in the hackmatch-diploma GitHub organisation. Separate repositories are available for frontend and backend development. The project README links to the YouTrack board.

Project organisation: https://github.com/orgs/hackmatch-diploma/repositories

Documentation and project overview: https://github.com/hackmatch-diploma/.github

Frontend: https://github.com/hackmatch-diploma/hackmatch-frontend

Backend: https://github.com/hackmatch-diploma/hackmatch-backend

YouTrack: https://hackmatch.youtrack.cloud/agiles/204-0/current

The development workflow links work items to an owner, acceptance criteria, deadline and relevant code or documentation changes. Feature branches and pull requests provide review points; release tags will identify submitted versions.

Risk assessment approach

Likelihood and impact are rated from 1 (low) to 3 (high). Their product determines priority: 6 to 9 is high, 3 to 4 medium, and 1 to 2 low. The ratings are initial estimates. Each risk has an owner, an observable trigger, preventive measures and a contingency response. Risks will be reviewed weekly and when a trigger occurs.

R1 Insufficient or inconsistent research data

Priority: 3 x 2 = 6 high. Owner: Elkham Mamedov. A small exploratory sample, non-participants and contradictory answers may produce misleading requirements. Trigger: inconsistent experience answers remain unresolved or no organiser is interviewed.

Mitigation: Validate responses with consent; recruit experienced participants and conduct 3-5 user discussions. Record the recruitment method and exclusions.

Contingency: Report preliminary results only; avoid subgroup claims; prioritise requirements confirmed by an organiser.

R2 Infeasible team constraints

Priority: 3 x 3 = 9 high. Owner: Ilnur Saipiyev. Team size, availability and role demand may conflict with the available pool. Trigger: the feasibility check cannot place all eligible participants.

Mitigation: Separate mandatory rules from preferences; validate role supply and size arithmetic before matching. Test small and odd participant pools.

Contingency: Leave affected participants explicitly unassigned and show the cause; let the organiser change optional criteria with confirmation. Never silently violate a mandatory rule.

R3 Incorrect or incomplete profiles

Priority: 3 x 2 = 6 high. Owner: Ilnur Saipiyev. Self-reported skill levels and missing availability may generate misleading recommendations. Trigger: required profile fields are missing or users reject suggestions because the data is wrong.

Mitigation: Use controlled skill labels, explicit unknown values, required availability and a profile confirmation step. Explain which fields drive the suggestion.

Contingency: Exclude incomplete profiles from automatic allocation, request corrections and offer manual team discovery.

R4 Poor matching outcomes or low trust

Priority: 2 x 3 = 6 high. Owner: Ilnur Saipiyev. Optimising one weighted score may leave weak teams or produce unwanted assignments. Trigger: low acceptance, reduced minimum role coverage or complaints about explanations.

Mitigation: Keep matching opt-in; expose reasons; compare minimum and mean coverage and acceptance against the baseline; test alternative weights.

Contingency: Revert to the baseline or organiser-assisted allocation; retain participant confirmation and reassignment.

R5 Authorisation failure or data exposure

Priority: 2 x 3 = 6 high. Owner: Ilnur Saipiyev. Missing event-level checks may expose profiles or permit score and team changes by the wrong user. Trigger: a cross-role or cross-event negative test succeeds.

Mitigation: Enforce access checks on every endpoint; minimise public profile fields; test judge assignments; keep secrets outside Git.

Contingency: Block the affected endpoint, revoke exposed credentials when applicable, assess affected records and repair before reopening access.

R6 Concurrent membership or deadline errors

Priority: 2 x 3 = 6 high. Owner: Ilnur Saipiyev. Simultaneous invitations or last-minute submissions may create duplicate membership, overfull teams or disputed deadlines. Trigger: a concurrency or deadline boundary test fails.

Mitigation: Use transactions and membership uniqueness rules, server-side time checks and repeatable integration tests. Display event timezone and submission confirmation.

Contingency: Suspend the affected workflow; reconcile records from the audit trail and backups; let the organiser resolve affected cases with a recorded decision.

R7 Integration and schedule overrun

Priority: 3 x 2 = 6 high. Owner: Elkham Mamedov. A three-person team may lose time when frontend and backend contracts diverge or matching expands in scope. Trigger: a core milestone slips by more than one week or an API contract remains unresolved.

Mitigation: Agree versioned API contracts; integrate one end-to-end flow early; implement the baseline before optimisation; review dependencies weekly.

Contingency: Freeze optional features, deliver the core registration-to-submission flow and revise remaining milestones with the supervisor.

R8 Deployment or database failure

Priority: 2 x 3 = 6 high. Owner: Ilnur Saipiyev. Hosting outages, failed migrations or missing backups may block the demonstration or lose submissions. Trigger: health checks fail, a migration fails or a backup cannot be restored.

Mitigation: Use a reproducible deployment, migration checks, health monitoring and database backups; rehearse a restore before the demonstration.

Contingency: Restore the last verified backup or run the prepared local demonstration with anonymised data; record the outage and verify integrity before resuming.

Milestones

The project proceeds through four milestones: validated requirements and user evidence; architecture, API contracts and interface design; an integrated registration-to-submission workflow with baseline matching; and evaluation, defect correction and release preparation. Dependencies include access to user representatives, consented profile data, an available deployment environment and supervisor review.

11. Supervisor Feedback

“The project addresses a relevant problem by proposing a centralized platform for hackathon management and multi-criteria student team formation. The team has demonstrated good progress in requirements analysis, literature review, technology selection and system architecture design. The proposed solution has a clear structure and practical potential; however, further work is required to complete implementation, integration and testing. Overall, the project has a solid foundation and is recommended for further development.” - Nurtas Nadirov.

References

Agarwal, J., Piatt, E., & Imbrie, P. K. (2022). Team formation in engineering classrooms using multi-criteria optimization with genetic algorithms. *2022 IEEE Frontiers in Education Conference (FIE)*, 1-6. https://doi.org/10.1109/FIE56618.2022.9962741

Forshaw, M., Alexandru, A., Bentley, C., González-Zelaya, V., Ajanovski, V., Bikanga Ada, M., Brooks, J., Burridge, J., Chao, A., Engineer, R., Glebova, O., Islam, T., Kiyohara, M., Ko, S.-H., Kristbergsson, E. S., Peltsverger, S., Russell, S., Samary, M. M., Steenbergen, M., & Wortmann, C. (2026). Fairness in Student Group Formation: Perspectives, Priorities, Compromises, Mechanisms, and Tooling. *ITiCSE-WGR - Publ. Working Group Rep. Innov. Technol. Comput. Sci. Educ.*, 217-276. https://doi.org/10.1145/3760545.3783973

Gómez-Zará, D., Andreoli, S., Dechurch, L. A., & Contractor, N. S. (2019). Discovering collaborators online: Assembling interdisciplinary teams online at an Argentinian University. *Cuadernos.Info*, (44), 21-41. https://doi.org/10.7764/cdi.44.1575

Hastings, E. M., Ojha, V., Austriaco, B. V., Karahalios, K., & Bailey, B. P. (2023). Composing team compositions: An examination of instructors' current algorithmic team formation practices. *Proceedings of the ACM on Human-Computer Interaction*, *7*(CSCW2), Article 305. https://doi.org/10.1145/3610096

Hevenstone, D., Endrissat, N., Lavrovsky, O., & Hümbelin, O. (2026). Designing for diversity: team formation and participatory policy design. *Policy Design and Practice*, *9*(2), 232-243. https://doi.org/10.1080/25741292.2025.2610868

Jadhav, A. V., Masrouri, M., & Qin, Z. (2026). ResearchConnect: An AI-Powered Platform for Interdisciplinary Research Team Formation and Ideation Development. *Advanced Intelligent Systems*, *8*(5). https://doi.org/10.1002/aisy.202501399

Kohli, S., Ramachandran, N., Tudor, A., Tumushabe, G., Hsu, O., & Ranade, G. (2023). Inclusive study group formation at scale. In *Proceedings of the 54th ACM Technical*

Symposium on Computer Science Education (Vol. 1, pp. 11-17). Association for Computing Machinery. https://doi.org/10.1145/3545945.3569885

Mendes, W., Richard, A., Tillo, T.-K., Pinto, G., Gama, K., & Nolte, A. (2022). Socio-technical constraints and affordances of virtual collaboration: A study of four online hackathons. *Proceedings of the ACM on Human-Computer Interaction*, *6*(CSCW2), Article 330, 1-32. https://doi.org/10.1145/3555221

Singh, P., Huynh, P. K., Nguyen, D., Le, T. Q., & Moreno, W. (2025). Leveraging Multicriteria Integer Programming Optimization for Effective Team Formation. *IEEE Transactions on Learning Technologies*, *18*, 72-84. https://doi.org/10.1109/TLT.2024.3401734

Vinella, F. L., Hu, J., Lykourentzou, I., & Masthoff, J. (2022). Crowdsourcing Team Formation With Worker-Centered Modeling. *Frontiers in Artificial Intelligence, 5*, Article 818562. https://doi.org/10.3389/frai.2022.818562

Wu, Y., Mo, W., & Liang, L. (2026). Reducing Intragroup Pressure in Heterogeneous Grouping for Collaborative Learning. *Proceedings of the 3rd International Conference on Educational Digitalization and Computer Science in the Guangdong-Hong Kong-Macao Greater Bay Area, EDCS '26*, 1326-1331. https://doi.org/10.1145/3828183.3828386

Xu, B., Lin, S., Zhu, Y., & Zhou, Y. (2026). Personality-Informed Team Formation for Collaborative Learning in Design Education: A Regression-Guided Framework. *Proceedings of the 2026 International Conference on Big Data and Informatization Education, ICBDIE '26*, 512-517. https://doi.org/10.1145/3806980.3807060